DevSecOps Workflow

Security-Integrated Software Delivery

Technical Documentation | Assessment Task 1

Name	Shreya V (192565018)
Course Nme	DevSecOps
Course code	CSA 5802

1. Introduction

DevSecOps integrates security into every stage of the Software Development Life Cycle (SDLC), replacing end-of-cycle security reviews with automated, continuous checks embedded directly in development and deployment pipelines. The result is faster delivery, lower remediation cost, and a measurable reduction in production vulnerabilities over time.

> *Security is not a final gate — it is a shared, automated quality attribute woven through every phase of delivery.*

Core Principles

Shift Left — security checks begin at planning and coding, not at release.

Automation — repeatable, tool-driven controls replace manual gatekeeping.

Shared Responsibility — developers, operations, and security engineers collaborate.

Continuous Feedback — every stage routes actionable signals back to earlier stages.

Measurable Improvement — vulnerability trends and mean-time-to-remediate are tracked.

2. DevSecOps Lifecycle — Five Phases

The pipeline is organised into five colour-coded phases. Sequential flow runs left to right; a continuous feedback mechanism returns signals from operations back to planning, forming a closed loop.

Phase	Concepts	Purpose
Plan & Design	Planning, Requirements, Threat Modeling, Secure Design	Define scope; identify and model risks before coding.
Develop & Commit	Secure Coding, SAST, Git, GitHub, Branch, Pull Request	Write code securely; manage changes via version control.
Build & Verify (CI)	Continuous Integration, SCA, Secret Scanning, DAST	Build and test every change; block vulnerabilities automatically.
Package & Release (CD)	Docker, Container Scanning, CD, IaC, Kubernetes, Deployment	Package, scan, and release into orchestrated environments.
Operate & Learn	Monitoring, Logging, SIEM, Incident Response, Feedback Loop	Observe production; route findings back into planning.

Phase Detail — Plan & Design

Planning sets security objectives (compliance targets, data classification, risk thresholds) before any code is written. Requirements capture non-functional security specifications — authentication strength, data protection, regulatory obligations. Threat Modeling uses STRIDE to enumerate attackers, vectors, and weaknesses, producing a prioritised mitigation list. Secure Design applies least-privilege and defence-in-depth principles to translate those findings into architecture decisions.

Phase Detail — Develop & Commit

Developers follow Secure Coding practices — input validation, parameterised queries, secure error handling — and commit to feature Branches in Git hosted on GitHub. SAST tools scan source in real time inside the IDE and as a pre-commit hook. Completed branches are proposed through Pull Requests, which enforce peer code review and trigger automated pipeline checks before any code reaches the main branch.

3. Build, Release & Operations

Phase Detail — Build & Verify (CI)

Every Pull Request triggers a Continuous Integration pipeline that runs three automated security gates in parallel: Software Composition Analysis (SCA) checks open-source dependencies against the National Vulnerability Database; Secret Scanning inspects commits for exposed API keys, tokens, or passwords; and Dynamic Application Security Testing (DAST) exercises a live staging build as an external attacker would. A failed gate blocks the merge, ensuring only vetted code progresses.

Phase Detail — Package & Release (CD)

Verified code is packaged into a Docker container image and passed to Container Scanning — a hard pipeline gate that inspects OS packages, image layers, and embedded secrets. Only images that pass this scan are promoted through Continuous Deployment. Infrastructure as Code (IaC) templates provision the Kubernetes cluster that orchestrates running containers, applying network segmentation and role-based access control at runtime. The final Deployment step releases the versioned, scanned artefact to production.

Phase Detail — Operate & Learn

Monitoring — tracks performance, availability, and security signals such as unusual traffic or repeated failed logins.

Logging — records timestamped events across applications, containers, and infrastructure for forensic reconstruction.

SIEM — aggregates and correlates logs, raising alerts on patterns no single source would reveal.

Incident Response — contains, investigates, and remediates confirmed security events; root-cause findings feed back into planning.

4. Reinforcement-Learning Feedback Loop

The Feedback Loop borrows reinforcement learning (RL) vocabulary to model how the DevSecOps pipeline improves over time. No machine-learning algorithm runs; the analogy frames security signals as rewards that drive measurable policy change.

RL Concept	DevSecOps Equivalent	What It Drives
Agent	Dev team / pipeline	Commits code, configures pipelines, responds to alerts.
Environment	Running app & infra	Generates observable results from every team action.
Reward Signal	Scan results, SIEM alerts	Positive = clean scans & stable production; Negative = vulnerabilities or incidents.
Policy Update	Updated coding standards	Recurring findings update Secure Coding guidelines and Threat Models — not just one-off patches.

Signal Stages & Speed

Fast (minutes) — SAST, SCA, and Secret Scanning return per-commit results attributable to a specific author.

Pre-release (hours) — Container Scanning and DAST validate the packaged artefact before it reaches production.

Production (ongoing) — Monitoring, Logging, and SIEM surface anomalies invisible to static tools.

Incident (days) — Post-incident root-cause analysis produces the richest signal — but also the most costly.

> *The loop only produces lasting improvement if findings update standards and future threat models — not just get individually patched. Recurring finding categories must change the guidelines governing the next planning cycle.*

5. Conclusion & References

DevSecOps is a closed, self-correcting system — not a linear pipeline that ends at deployment. Security is distributed across all five phases, and every phase generates signals that the Feedback Loop returns to developers and architects. Over successive iterations, the volume and severity of new findings trend downward, producing measurable, compounding improvement in application security.

References

- OWASP Foundation — OWASP Top 10 (owasp.org)
- OWASP Foundation — Software Assurance Maturity Model (SAMM)
- OWASP Foundation — DevSecOps Guideline
- NIST SP 800-218 — Secure Software Development Framework (SSDF)
- National Vulnerability Database (NVD) — nvd.nist.gov